Exercise 2: First Models — CTEs and the ref() Tag

Objective

Load real data as seeds, build a source layer with CTEs, and wire models together using `ref()` to form a dependency graph.

Prerequisites

- Exercise 1 completed (dbt installed, `airbnb/` project created, connection working)

About the Dataset

We will model a mini **Airbnb Barcelona** dataset: listings, hosts, and reviews. This mirrors real-world dbt projects where you transform raw source tables into clean, analytics-ready models.

The three layers we will build today:

Layer	Folder	Purpose
Raw	`seeds/`	CSV files loaded as-is into the database
Staging (src)	`models/src/`	Clean, rename, cast — one model per source table
Marts	`models/marts/`	Business models combining multiple sources

Step 1: Remove the example models

Delete the boilerplate that `dbt init` created:

```
rm -rf models/example/
```

Step 2: Create the seed CSV files

Seeds are CSV files that dbt loads directly into your database. They are ideal for **static reference data** or, as here, for loading raw datasets during learning.

Create the `seeds/` directory and add three files:

`seeds/raw_listings.csv`

```
id,name,host_id,neighbourhood,room_type,minimum_nights,price,created_at,updated_at
1001,Cozy Studio in Gracia,101,Gracia,Entire home/apt,2,85,2024-01-01,2025-01-15
1002,Sunny Room near Park,102,Gracia,Private room,1,45,2024-01-02,2025-02-20
1003,Modern Flat in Eixample,101,Eixample,Entire home/apt,3,120,2024-01-03,2025-01-10
1004,Charming Apartment Gothic,103,Barri Gotic,Entire home/apt,2,95,2024-01-04,2025-
03-01
1005,Beachfront Studio Barceloneta,102,Barceloneta,Entire home/apt,1,110,2024-01-
05,2025-02-01
1006,Loft in Poblenou,104,Poblenou,Entire home/apt,3,75,2024-01-06,2025-01-20
```

```
1007,Budget Room Sagrada Familia,103,Eixample,Private room,1,35,2024-01-07,2025-03-15
1008,Penthouse with Terrace,104,Eixample,Entire home/apt,2,200,2024-01-08,2025-01-05
```

`seeds/raw_hosts.csv`

```
id,name,is_superhost,created_at,updated_at
101,Maria Garcia,t,2023-01-01,2025-01-10
102,Carlos Lopez,f,2023-02-15,2025-02-20
103,Ana Fernandez,t,2023-03-01,2025-03-01
104,Juan Martinez,f,2023-04-10,2025-01-25
```

`seeds/raw_reviews.csv`

```
listing_id,reviewer_name,rating,review_date
1001,John Smith,5,2025-01-20
1001,Sarah Jones,4,2025-02-10
1002,Mike Brown,5,2025-01-15
1003,Emma Wilson,3,2025-02-05
1004,Tom Davis,5,2025-03-05
1005,Alice Lee,4,2025-01-30
1006,Bob Taylor,5,2025-02-15
1007,Clara Wright,4,2025-03-10
1008,David Martin,5,2025-01-08
1001,Pierre Dupont,5,2025-03-01
```

Step 3: Load the seeds into DuckDB

```
dbt seed
```

dbt reads each CSV and creates a table in DuckDB:

```
1 of 3 START seed file dev.raw_hosts ........ [RUN]
1 of 3 OK loaded seed file dev.raw_hosts .... [INSERT 4 in 0.05s]
2 of 3 START seed file dev.raw_listings ..... [RUN]
2 of 3 OK loaded seed file dev.raw_listings . [INSERT 8 in 0.04s]
3 of 3 START seed file dev.raw_reviews ...... [RUN]
3 of 3 OK loaded seed file dev.raw_reviews .. [INSERT 10 in 0.04s]
```

Verify in Python:

```python
import duckdb

conn = duckdb.connect("airbnb.duckdb")
print(conn.execute("SELECT * FROM raw_listings").df())
conn.close()
```

Step 4: Create the staging layer — src_listings

Create `models/src/src_listings.sql`:

```
WITH raw AS (
    SELECT * FROM {{ ref('raw_listings') }}
)

SELECT
    id                                      AS listing_id,
    name                                    AS listing_name,
    host_id,
    neighbourhood,
    room_type,
    minimum_nights,
    CAST(price AS DECIMAL(10, 2))           AS price_usd,
    CAST(created_at AS DATE)                AS created_at,
    CAST(updated_at AS DATE)                AS updated_at
FROM raw
```

What is a CTE?

A **Common Table Expression** (CTE) is a named subquery declared with `WITH`. dbt strongly recommends CTE-based models because they are:

- **Readable**: each transformation step has a name and a clear purpose
- **Debuggable**: you can comment out later CTEs and inspect intermediate results
- **Composable**: you can chain multiple CTEs, each building on the previous one

Convention: always start with a CTE called `raw` or `source` that selects from the raw table unchanged. This makes it easy to swap the source without rewriting logic.

Step 5: Create src_hosts and src_reviews

`models/src/src_hosts.sql`

```
WITH raw AS (
    SELECT * FROM {{ ref('raw_hosts') }}
)

SELECT
    id                                      AS host_id,
    name                                    AS host_name,
    CASE WHEN is_superhost = 't' THEN TRUE
         ELSE FALSE END                     AS is_superhost,
    CAST(created_at AS DATE)                AS created_at,
    CAST(updated_at AS DATE)                AS updated_at
FROM raw
```

`models/src/src_reviews.sql`

```sql
WITH raw AS (
    SELECT * FROM {{ ref('raw_reviews') }}
)

SELECT
    listing_id,
    reviewer_name,
    CAST(rating AS INTEGER)                 AS rating,
    CAST(review_date AS DATE)               AS review_date
FROM raw
```

Step 6: Create a mart model — fct_listings

Now wire the staging models together using `ref()`. Create `models/marts/fct_listings.sql`:

```sql
WITH listings AS (
    SELECT * FROM {{ ref('src_listings') }}
),

hosts AS (
    SELECT * FROM {{ ref('src_hosts') }}
),

final AS (
    SELECT
        l.listing_id,
        l.listing_name,
        l.neighbourhood,
        l.room_type,
        l.minimum_nights,
        l.price_usd,
        h.host_name,
        h.is_superhost,
        l.created_at,
        l.updated_at
    FROM listings l
    LEFT JOIN hosts h ON l.host_id = h.host_id
)

SELECT * FROM final
```

Notice: `fct_listings` never mentions `raw_listings` or `raw_hosts` directly. It only references the clean staging models. This is the dbt contract: each model has exactly one upstream dependency per source.

Step 7: Run the models

```
dbt run
```

dbt resolves the dependency order automatically:

```
1 of 6 START seed file dev.raw_hosts ......... OK
2 of 6 START seed file dev.raw_listings ....... OK
3 of 6 START seed file dev.raw_reviews ........ OK
4 of 6 START sql view model dev.src_listings .. OK
5 of 6 START sql view model dev.src_hosts ..... OK
6 of 6 START sql view model dev.fct_listings .. OK
```

Step 8: Visualize the DAG

```
dbt docs generate
dbt docs serve
```

Open http://localhost:8080 in your browser. Click the **graph icon** at the bottom right. You will see the full lineage:

```
raw_listings  ──►  src_listings  ─┐
                                  ├──►  fct_listings
raw_hosts     ──►  src_hosts     ─┘

raw_reviews   ──►  src_reviews
```

This is the **DAG (Directed Acyclic Graph)** that dbt builds from your `ref()` calls. dbt uses it to determine execution order and to know which models to rebuild when something upstream changes.

Step 9: Query the final model

```python
import duckdb

conn = duckdb.connect("airbnb.duckdb")
print(conn.execute("""
    SELECT listing_name, neighbourhood, price_usd, host_name, is_superhost
    FROM fct_listings
    ORDER BY price_usd DESC
""").df())
conn.close()
```

You should see 8 rows with listing and host information joined.

Key Concepts

Concept	What it means

Seed	A CSV file loaded by `dbt seed`. Good for static/reference data.
CTE	Named subquery with `WITH`. dbt models use one CTE per transformation step.
`ref()`	References another model. Builds the DAG. Resolves to the real table/view name.
Staging layer	Cleans raw data: renames columns, casts types, no business logic.
Mart layer	Combines staging models into business-facing datasets.

Questions

1. Why do staging models (`src_*`) never contain joins between tables?
2. What happens if you rename `raw_listings.csv` to something else without updating `src_listings.sql` ?
3. `fct_listings` uses `LEFT JOIN` . What rows would be lost with an `INNER JOIN` ?